IT305 LAB

Fault-Tolerant Topic-Based File Distribution Service

Final Code and Result Analysis

Groups 064 and 091

Purva Kaneria	20241171
Soham Doshi	202401208
Shaurya Shekhawat	202401197
Om Patel	202401439
Aditya Vaish	202401401
Tirth Patel	202401227

Report generated from repository state and measured validation runs on 22 October 2026.

Executive Summary

The project implements a C11/POSIX TCP service that maps topic names to directories and transfers every regular file in a requested topic. Part I compares sequential and thread-per-client servers. Part II adds probabilistic disconnection, restart-only recovery, durable checkpoint resumption, and parallel byte-range transfer. The implementation uses a shared 28-byte frame header, bounded payloads, explicit offsets, SHA-256 integrity checks, safe temporary files, and strict command-line validation.

Current code status: all programs compile with GCC using -Wall -Wextra -Wpedantic -Werror; five unit suites and four TCP end-to-end suites pass; AddressSanitizer and UndefinedBehaviorSanitizer pass across unit tests and every end-to-end case. The generated submission layout also builds each required case independently.

Evidence boundary: the performance figures in this report are real same-host WSL2 loopback measurements on a 12.58 MB validation workload. They verify the instrumentation and relative behavior, but they are not the assignment's required Animals-10 experiment using one server computer and two client computers. That final physical experiment remains the only unresolved submission requirement.

Team Contributions

Member	Primary contribution
Purva Kaneria	Enhanced Case 2: parallel range design, client/server implementation, experiment harness, plots and analysis.
Soham Doshi	Part I topic discovery, safe path handling, sequential and multi-threaded servers.
Shaurya Shekhawat	Case 2 server session store, durable checkpoints, RESUME/RESUME_OK protocol and fallbacks.
Om Patel	Shared framing/serialization/socket layer, integration, protocol documentation, hardening, sanitizer and final-package checks.
Aditya Vaish	Common client, temporary-file publication, size/SHA-256 validation, client CLI and initial packaging layout.
Tirth Patel	Case 1 fault injection, restart client, exact metrics, experiment and plotting workflow.

Repository history and module ownership support these contribution lines; every member remains jointly responsible for the whole system and viva.

1. Requirements and Compliance

Requirement	Implemented evidence	Status
Part I sequential and concurrent servers	server_single and server_multi share validated transfer logic	Pass
Required CLI; no fixed path/IP/port	Strict positional parsing; dataset root, port, host and output are runtime arguments	Pass
Case 1 restart after disconnect	Fresh GET and transfer ID; failed attempt output discarded	Pass
Case 2 checkpoint resume	Atomic client checkpoint plus validated in-memory server session	Pass
Enhanced parallel connections	N workers request non-overlapping ranges and reassemble with pwrite	Pass
Per-case Makefile builds server/client	Generated Part1 and three Part2 folders build exactly two executables	Pass
Raw logs and graphs for all cases	CSV and PNG outputs exist and are reproducible	Local validation
Three-computer Animals-10 experiment	Procedure is documented; current repository does not contain those final measurements	Pending

Required interfaces

```
Part I: ./server <port> <topics_root_dir>
Part II: ./server <port> <topics_root_dir> <failure_probability>
All clients: ./client <server_ip> <server_port> <topic> <output_dir>
```

The enhanced client accepts an optional trailing parallel-connection count while preserving the required four-argument client invocation.

2. Architecture and Wire Protocol

Layer	Responsibility
common/	Frame encoding, network-order integers, full-buffer send/receive, CLI parsing, Case 2 and range payload codecs.
server/	Topic manifest construction, safe file access, sequential/concurrent service, injected faults, session state and range workers.
client/	Manifest validation, temporary-file writes, SHA-256 publication, restart/resume clients and parallel reassembly.
tests/	Unit boundaries, malformed peers, real TCP transfers, disconnects, restart fallback, modified-manifest fallback and concurrency.
scripts/	Repeatable experiments, summaries, plotting and creation of the graded directory layout.

Frame format

Every message begins with version (1 byte), type (1), flags (2), payload length (8, network order), and transfer ID (16): 28 bytes total. Receivers loop until the full header and declared payload arrive. Payload limits are enforced before allocation, and every DATA frame carries a file index and absolute offset.

Phase	Messages
Base transfer	GET, MANIFEST, FILE_START, DATA, FILE_END, ACK, COMPLETE, ERROR
Checkpoint resume	RESUME, RESUME_OK plus base transfer messages
Parallel ranges	ASSIGN_RANGE, RANGE_READY, DATA, ACK_RANGE, RANGE_END

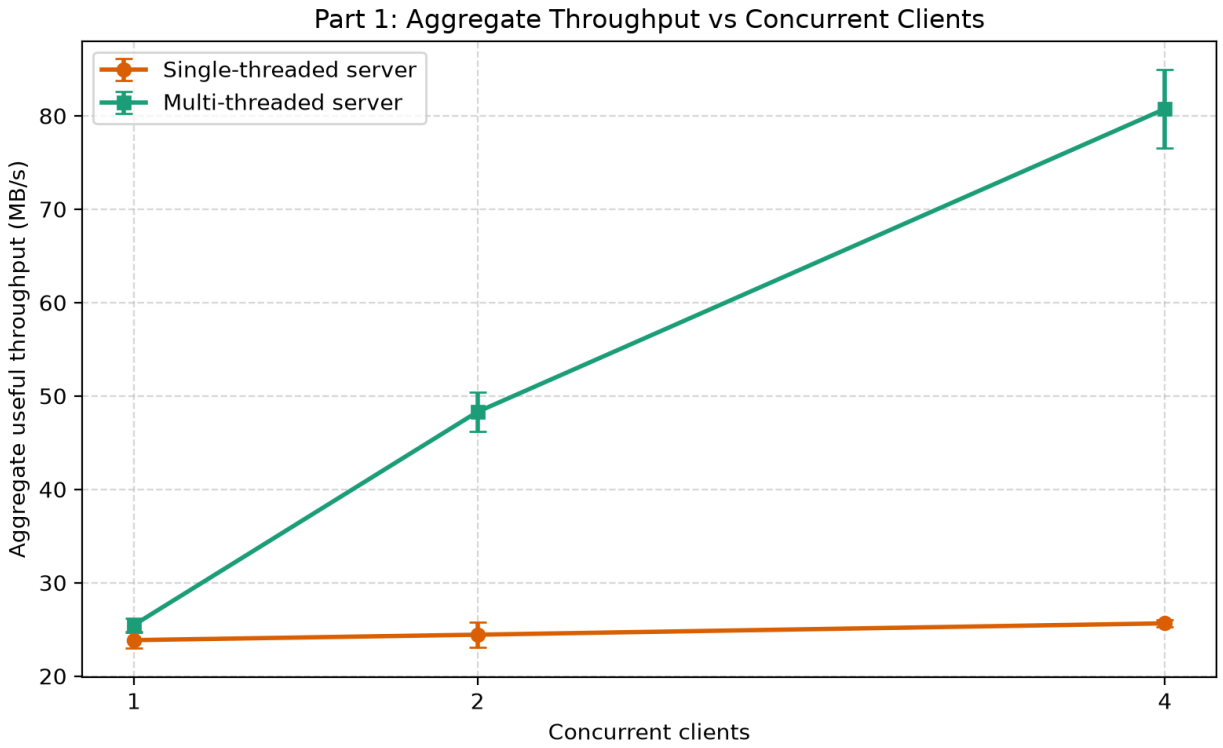
Security and correctness controls

Topic and file names reject traversal, separators, absolute components, control characters and duplicates. The server serves only regular files beneath the selected topic. The client writes .part files, flushes them, verifies exact size and SHA-256, then atomically renames. Socket I/O uses deadlines and treats short frames as interruption. Manifest identities prevent resuming against changed server content.

3. Part I - Basic Topic Distribution

The single-threaded server accepts one client and completes its request before returning to accept. The multi-threaded server allocates a per-client context and detaches one worker thread for every accepted connection. Both modes use the same sorted manifest, framing and file-transfer implementation, so the comparison isolates scheduling rather than protocol differences.

Mode	Clients	Mean MB/s	Std. dev.
Single	1	23.85	0.87
Single	2	24.43	1.31
Single	4	25.65	0.34
Multi	1	25.47	0.76
Multi	2	48.30	2.11
Multi	4	80.74	4.17

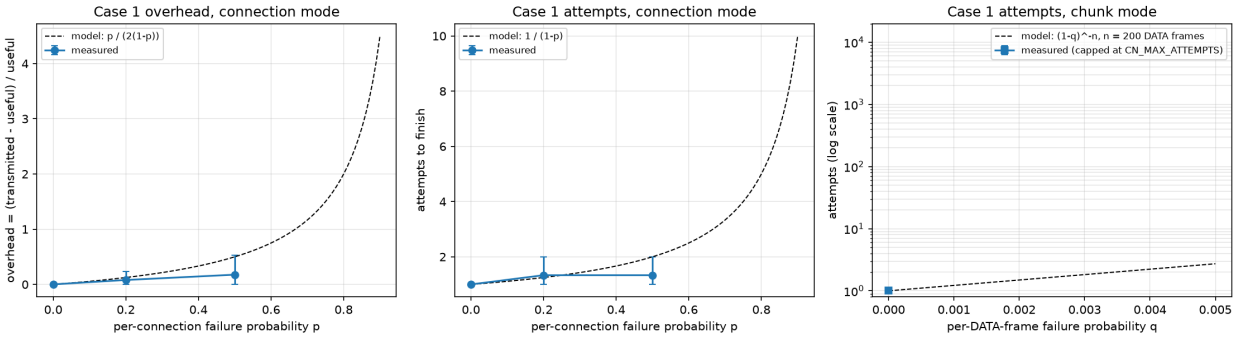


In loopback validation, sequential throughput stayed near 24-26 MB/s as client count increased because requests were serialized. The multi-threaded server rose from 25.47 MB/s at one client to 80.74 MB/s at four clients, a 3.17x aggregate increase. This is a host-local concurrency result, not a lab-network capacity claim.

4. Part II Case 1 - Restart Without Session State

After a manifest is sent, the server either completes normally or cuts the stream at a seeded byte position selected with probability p. A cut can use FIN or TCP RST. The client retains no progress: it removes partial output, reconnects, sends a new GET with a new transfer ID, and restarts from byte zero. Exact counters distinguish useful, redundant and total transmitted bytes.

Failure p	Mean goodput MB/s	Std. dev.	Runs
0	21.54	3.14	3
0.2	19.78	3.05	3
0.5	25.63	8.25	3

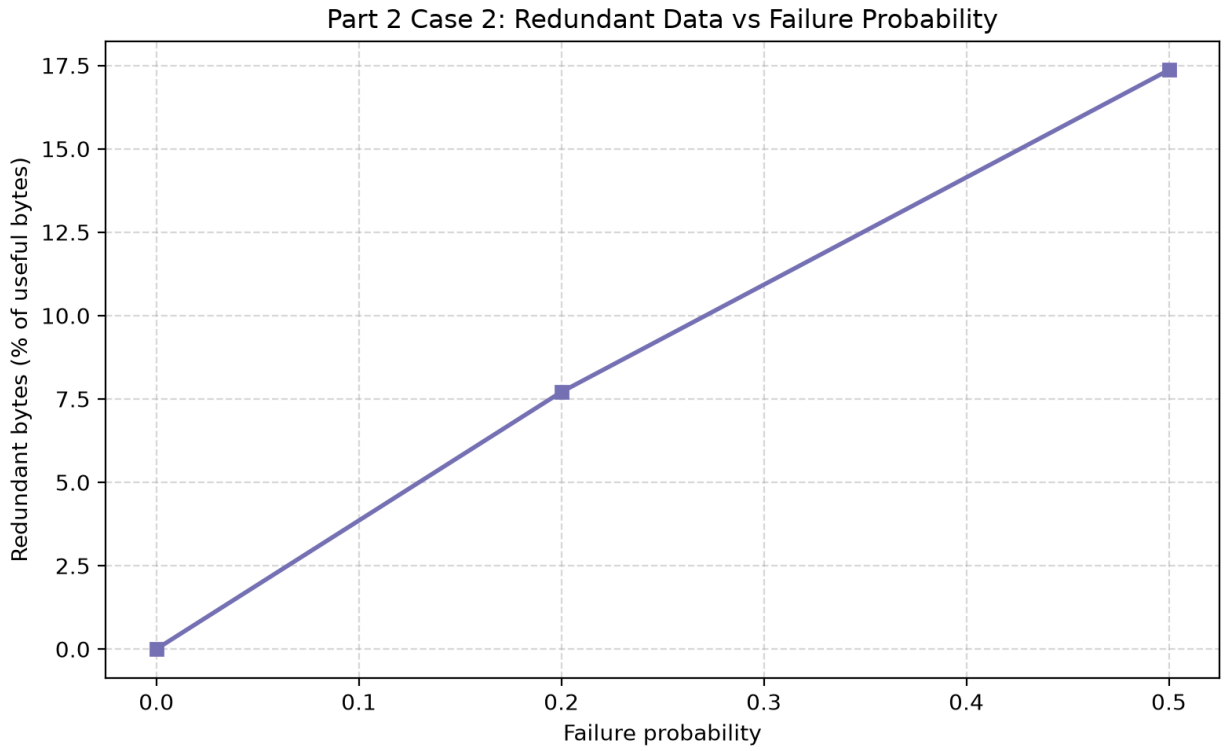


The p=0.5 mean goodput is noisy because two of three seeded runs had no cut. The byte metric remains diagnostic: mean redundant data rose to 2.18 MB. Theory predicts geometric attempts and expected redundant data pD/(2(1-p)); more repetitions are needed for stable high-p timing.

5. Part II Case 2 - Checkpointed Resume

The client creates a 16-byte transfer ID and persists topic, manifest identity, current file, contiguous offset and completed-file digests to an atomic checkpoint. The server stores only acknowledged progress in a mutex-protected session table. On reconnect, RESUME is accepted only when transfer ID, topic, manifest digest, completed hashes and requested offset agree with server state. Missing or stale server state safely falls back to a clean GET.

Failure p	Mean goodput MB/s	Std. dev.	Runs
0	2.51	0.05	3
0.2	2.37	0.30	3
0.5	2.24	0.61	3

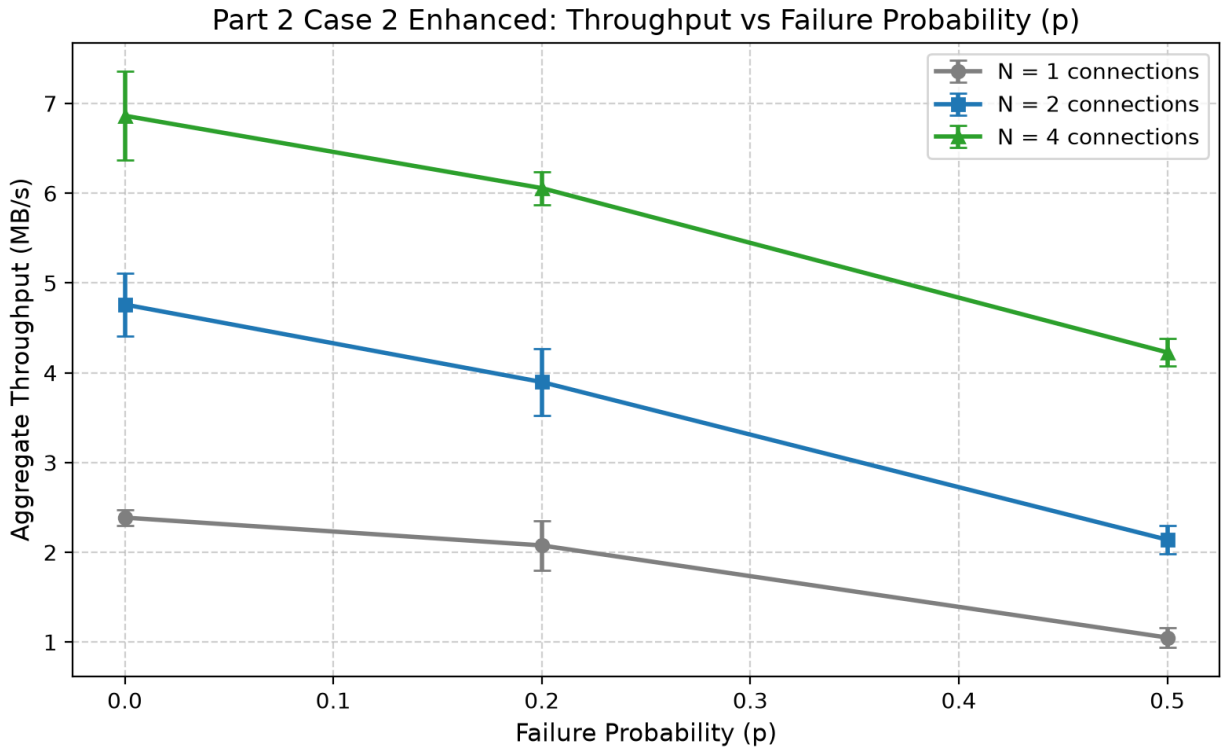


Mean redundant data increased with p but remained bounded by acknowledged progress rather than requiring every completed file to be retransmitted. The validation environment makes durable fsync checkpoints expensive, so Case 2 goodput is lower than Case 1 in these short loopback runs. This is a real implementation tradeoff: durability reduces re-sent bytes but adds synchronous storage work.

6. Part II Case 2 Enhanced - Parallel Ranges

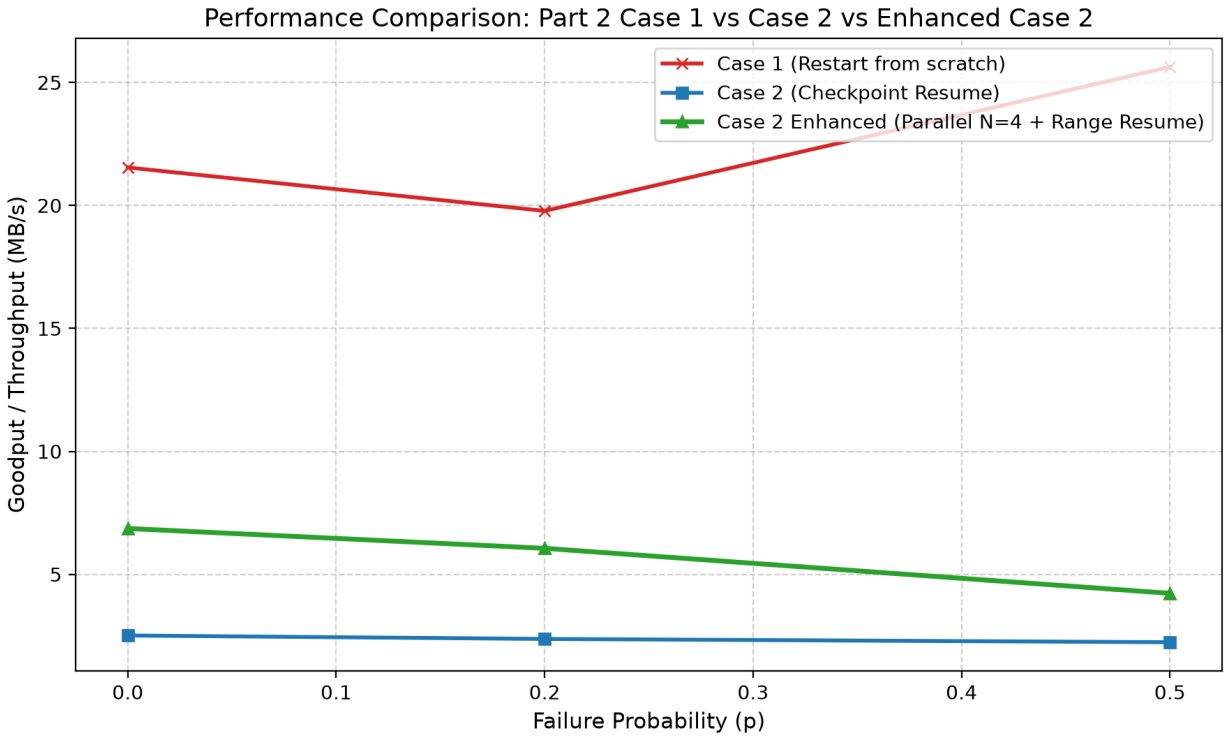
The enhanced client obtains the manifest, partitions files into disjoint half-open ranges, preallocates temporary files, and schedules ranges across N worker connections. Workers write with pwrite at explicit offsets, so independent ranges do not race on a shared file position. When a connection drops, only that range reconnects from its current offset; completed ranges and other workers continue.

p	N=1 MB/s	N=2 MB/s	N=4 MB/s
0	2.39	4.76	6.86
0.2	2.08	3.90	6.06
0.5	1.05	2.14	4.23



At p=0, N=4 reached 6.86 MB/s versus 2.39 MB/s for N=1 (2.88x). At p=0.2, the measured speedup was 2.92x. Throughput declined with faults, but unaffected workers continued. No complete DATA payload was received twice in this measured sweep, so redundant payload bytes were zero while control overhead increased slightly.

7. Cross-Case Analysis



All three Part II curves above use the same 12.58 MB workload and failure points. Case 1 is fastest in this local environment because it avoids durable checkpoint fsync cost and most three-run seeds completed without interruption. Case 2 trades throughput for durable, validated progress. Enhanced Case 2 recovers much of that cost through parallelism. The graph must not be generalized beyond this workload or used as the required three-computer result.

Metrics

Metric	Definition
Useful bytes	Unique file bytes in the final verified output tree.
Transmitted bytes	All server-to-client bytes observed across every attempt.
Redundant bytes	Complete DATA payload bytes received beyond useful bytes.
Wasted bytes	All bytes received on failed attempts.
Goodput	Useful bytes divided by end-to-end completion time.
Overhead	(transmitted bytes - useful bytes) / useful bytes.

8. Verification

Check	Observed result
Strict normal build	GCC 13.3.0; all targets compile with warnings promoted to errors.
Unit suites	Protocol, client, fault injection, Case 2 and enhanced range suites pass.
TCP end-to-end	Part I, Case 1, Case 2 and Enhanced suites pass.
Fault behavior	FIN and RST cuts, retries, resume, server restart, changed manifest and invalid topic covered.
Concurrency	Six Case 1 clients, four Case 2 clients and multiple enhanced range workers verified.
Sanitizers	AddressSanitizer and UndefinedBehaviorSanitizer pass for unit and all end-to-end suites.

Check	Observed result
Submission builds	Part1 plus all three Part2 folders independently produce executable server and client.

Representative commands

```
make clean && make -j2 all
make test && make e2e && make e2e_case1
make e2e_case2 && make e2e_enhanced
make submission
```

The sanitizer configuration additionally used -fsanitize=address,undefined and -fno-omit-frame-pointer for every binary.

9. Deviations, Limitations and Final Procedure

Item	Final implementation / limitation
Part I packaging	Part1/ builds multi-threaded server by default; make single rebuilds the same required ./server name.
Server session persistence	Session state is intentionally in memory. A server restart triggers safe client fallback to full GET.
Checkpoint cost	Durable fsync-based checkpoints reduce re-sent data but are expensive on this validation host.
Enhanced persistence	Range progress resumes per worker connection; final files publish only after complete SHA-256 verification.
Performance evidence	Current results are local WSL2 validation, not the mandatory lab three-computer Animals-10 experiment.

Required final physical run

1. Copy the same commit to three Linux computers. 2. Put the Animals-10 raw-img root on the server computer. 3. Run the server on one machine and concurrent clients on the other two. 4. Repeat matched Part I and Part II sweeps, recording commit hash, host specifications, topic size, client count, p, N and seed. 5. Copy the raw CSVs into Results/, regenerate every graph, regenerate this PDF, then rerun make submission and inspect the extracted archive.

Do not relabel the current loopback results as three-computer measurements. The code is submission-grade; the remaining evidence must come from the required physical setup.

10. Conclusion

The implementation satisfies the functional architecture of all four cases and has strong automated evidence for framing, integrity, safety, interruption handling, resumption and concurrency. Part I scales with concurrent clients on the validation host; Case 1 exposes restart waste; Case 2 bounds progress loss using validated checkpoints; and Enhanced Case 2 gains throughput by allowing independent ranges to proceed in parallel. Final submission should occur only after the team replaces or augments the local validation figures with the required three-computer Animals-10 experiment.

References

1. IT305 Lab: Socket Programming Project - Fault Tolerant Topic-based File Distribution Service.
2. Design_Document_Group_064_091_.pdf.
3. POSIX pthread and socket APIs; OpenSSL libcrypto SHA-256 implementation.